

ICS2121, Métodos de Optimización 2025-2

Una breve introducción a las conexiones IA-ML-Optimización

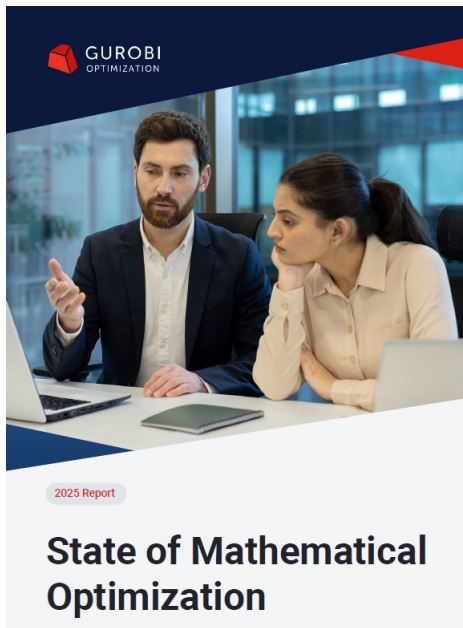
- Hace 20 años, nadie hablaba mucho de Inteligencia Artificial (IA), fuera del área específica.
- (El tema es muy antiguo, data de los años 1950)
- Sí se hablaba algo de Machine Learning (ML), había técnicas para problemas específicos.
- En el área de “negocios” se hablaba bastante de “Minería de Datos”
- Hace 10 años, ya se hablaba bastante de IA, pero se pensaba que el desarrollo fuerte tomaría muchos años
- Hoy, todos estamos “impresionados” por lo que ocurre con la IA...

- La Investigación Operacional, en particular la Optimización, y la IA y ML han sido, tradicionalmente, comunidades separadas.
- (IA y ML suelen identificarse en Ciencias de la Computación)
- Aunque casi todos los algoritmos de IA y ML tienen “en su corazón” un problema de optimización.
- Es decir, la optimización, parte de la Investigación Operacional, “ayuda” (mucho) a la IA.

- Pero, lo que nos interesa ahora es la dirección inversa:
- ¿De qué forma las herramientas de ML e IA ayudan a “optimizar” o, más en general, ayudan a abordar problemas de la Investigación Operacional?
- Esto es un tema bastante emergente, ha comenzado a explorarse apenas desde hace unos 10 años, y ahora están comenzando a haber muchos desarrollos.
- Lo que comentaré está basado, en parte, en:
 - ▶ Temas levantados por Isidora Carmona, Francisca Correa, Javiera Santibañez y Sebastián Sepúlveda, actualmente en investigación IPRE.
 - ▶ Benigio, Y., A. Lodi, A. Prouvost, Machine Learning for Combinatorial Optimization: A methodological tour d’horizon, European Journal of Operations Research 290 (2021) pp 405-421.
 - ▶ Mandi et al., Decision-Focused Learning: foundations, state of the art, benchmark and future opportunities.
 - ▶ Otras referencias.

Algo reciente:

- Reporte de Gurobi Optimization sobre el estado de la Optimización, 2025:



Algo reciente:

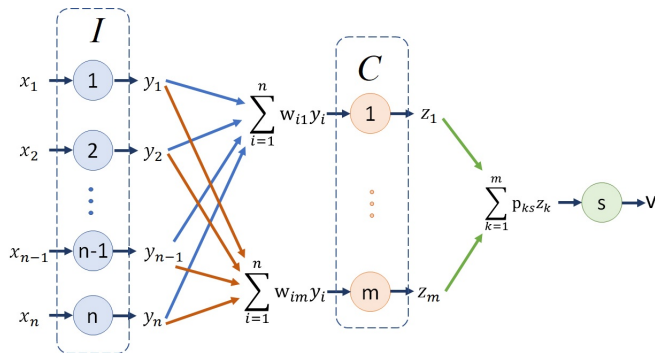
- El reporte destaca que Machine Learning en Optimización es ya una metodología madura.
- Pero que el gran salto viene con el uso de IA Generativa.
- Según el reporte, aún hay poco conocimiento de cómo apalancar GenIA con Optimización.
- Aunque 30% de los que respondieron la encuesta ven ya muchas oportunidades...

- Machine Learning es bastante conocido en estos tiempos.
- Proporciona técnicas para, fundamentalmente, analizar grandes cantidades de datos.
- Esta “analítica” puede ser:
 - ▶ Descriptiva: qué dicen los datos
 - ▶ Predictiva: qué podría pasar, sobre la base de los datos
 - ▶ Prescriptiva: qué decisiones deberíamos tomar, según el análisis de los datos.
- Las dos primeras se mezclan con la estadística “tradicional”...
- La tercera característica es más potente e interesante.

- IA, por otro lado, apunta a sistemas computacionales con capacidad de “aprendizaje” y “razonamiento” profundo.
- Es decir, cosas que, habitualmente, atribuimos a nuestra inteligencia.
- Algunos de los temas más conocidos en esto son:
- Aprendizaje Reforzado (Reinforcement Learning), especialmente el “Deep”.
- Inteligencia Artificial Generativa (GAI), particularmente los Large Language Models (LLM)

ML e IA:

- En ML existen muchas metodologías.
- Redes Neuronales, clasificación (SVM), Decision Trees, Random Forests, XGBoost, etc.
- Las Redes Neuronales suelen ser el modelo de cálculo más comentado, recordemos el ejemplo de la introducción:



- Los parámetros del modelo de cálculo son estimados (“entrenados”) a partir de datos.
- Una de las “gracias” del modelo es que permite “descubrir” relaciones no lineales entre input y output.

- ¿Qué hace un LLM? (parte de lo la Inteligencia Artificial Generativa)
- Implementa un modelo masivo de redes neuronales complejas, apoyado por aprendizaje reforzado, que busca combinar patrones de texto que tienen sentido con alta probabilidad.
- Las versiones recientes (GTP4 y GTP5) usan “agentes” para abordar tareas especializadas.



(Figura de S. Sepúlveda, 2025)

Reinforcement Learning (RL)

- Ya habíamos conversado algo de esto cuando hablamos de Programación Dinámica (PD).
- Vimos como el RL permite abordar, en forma aproximada, la resolución de PD de un modelo de decisiones Markoviano.
- Recordemos lo que era un Proceso de Decisión Markoviano: el sistema puede estar en un estado $i \in \{1, \dots, N\}$ y el pasar de un estado a otro de una etapa a la siguiente, depende de una probabilidad de transición y de la acción que se tome en la etapa:

$$p^k(x_k, i, j) = \text{Prob}(s_{k+1} = j / s_k = i, \text{acción} = x_k)$$

- La Ecuación de Bellman del sistema es:

$$J_k(i) = \min_{x_k \in U_k(i)} \left\{ g_k(i, x_k) + \sum_{j=1}^N p^k(x_k, i, j) J_{k+1}(j) \right\}$$

Reinforcement Learning (RL)

- Resolver, en forma exacta, esto es muy complejo debido a la explosión en el número de estados.
- El problema se transforma en:

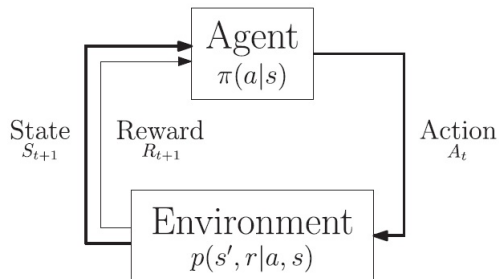
$$J_k(i) = \min_{x_k \in U_k(i)} \{g_k(i, x_k) + Q(x_k, i)\}$$

donde $Q(x_k, i)$ es una **aproximación** a $\sum_{j=1}^N p^k(x_k, i, j) J_{k+1}(j)$.

- Una forma de construir esta aproximación es:
 - 1 Simular una serie de escenarios (estados) futuros y obtener valores $Q(x_k, i)$
 - 2 Alimentar este conjunto de escenarios y los resultados, en una red neuronal para que “aprenda” el comportamiento del futuro.
 - 3 Alimentar, en “tiempo real”, los resultados reales.

Reinforcement Learning (RL)

- El RL es mucho más que su aplicación a Sistemas de Decisión Markovianos.



(imagen de Bengio et als(2021), adaptada de Sutton y Barto(2018))

- Un sistema de RL fue el que derrotó, en el juego del GO en 2016, a un maestro internacional, y desde ahí muchos comenzaron a prestar más atención al tema.

- ML ya ha estado siendo usada en Optimización desde hace algunos años.
- Por supuesto, un uso directo es el obtener mejores estimaciones de los parámetros de un modelo de optimización, a partir de datos, y después optimizar (“Predict then Optimize”).
- Pero otros usos muy interesantes se relacionan a mejorar el desempeño de algoritmos.

Aplicaciones en Programación Entera

- Consideremos el siguiente problema de optimización entera:

$$\begin{aligned} z^*(c) = \min \quad & f(x, c) \\ \text{s.a.} \quad & g(x, c) \leq 0 \\ & x \text{ entero} \end{aligned}$$

donde c son parámetros del problema.

- Lo típico, por supuesto, es que f y las restricciones g sean lineales.
- Estos son problemas conocidos como difíciles (NP-completos).
- ¿Cómo lo resolvemos?
- Con Branch & Bound, que ya hemos estudiado.
- Pero, recordemos una decisión importante en el algoritmo: qué variable (fraccionaria) elegir para ramificar.
- “Strong Branching” es una idea que se implementa frecuentemente: “probar” ramificaciones y estimar cuánto cambia la función objetivo.

Aprender del Strong Branching

- Pero, supongamos que almacenamos los resultados de la ramificación, diferenciando aquellos que han sido exitosos (encuentran soluciones enteras o cambian el valor significativamente) y los que no, a medida que avanza el proceso de búsqueda.
- En poco tiempo podemos tener un conjunto de datos de resultados de ramificación muy grande.
- Una idea: usar un método de **clasificación o de predicción** que permita “aprender” de esos datos y construir una regla de ramificación que se pueda usar en las iteraciones siguientes.
- La clasificación se puede hacer con un método de SVM (Support Vector Machine) u otro, de los disponibles en Machine Learning.
- También se puede “aprender” algún conjunto de variables que resultan en ramificaciones más exitosas (en términos de mejora en función objetivo).
- Esto se puede hacer con una Red Neuronal o un modelo de RL.

Pero el problema es difícil...

- Supongamos que un caso del problema sigue siendo muy difícil, aún para un B&B avanzado.
- Entonces, iríamos por una heurística, por ejemplo, Simulated Annealing o Búsqueda Tabú.
- Una etapa clave en estas y otras técnicas de búsqueda local es la identificación de un vecino de la solución actual.
- Supongamos que avanzamos en la búsqueda y vamos guardando la información de las soluciones vecinas y su impacto en la búsqueda.
- Después de varias iteraciones se dispondrá de un conjunto de historia de la búsqueda.
- Una Red Neuronal o un clasificador puede identificar estructuras o patrones en la solución que hagan favorable el avance de la búsqueda, y esto permite reducir las alternativas a explorar.

RL puede ayudar a la búsqueda local...

- También se puede aprender dinámicamente durante la búsqueda.
- Esto se puede hacer, combinando la heurística con Reinforcement Learning.
- Por ejemplo, si usamos Simulated Annealing con el Problema del Vendedor Viajero, podemos usar 2-Opt para construir vecinos.
- Supongamos que la red tiene m nodos y una solución cualquiera en la iteración t la denotamos por

$$s^t = \{e_1^t, e_2^t, e_3^t, \dots, e_m^t\}$$

donde e_j^t identifica el j -ésimo arco del circuito.

- 2-Opt elije al azar dos arcos no adyacentes del circuito: e_j^t y e_k^t , y los reemplaza por los “cruzados”: $\bar{e}_j$ y $\bar{e}_k$ para crear la nueva solución s^{t+1} .
- Denotemos por $f(s)$ el largo de una solución de circuito de vendedor viajero.

RL puede ayudar a la búsqueda local...

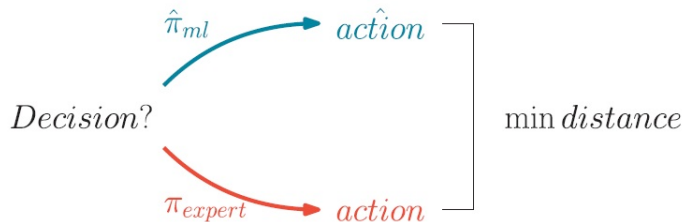
- Se puede construir un MDP (Markovian Decision Process) de la siguiente forma:
- **Etapas:** son cada una de las iteraciones.
- **Estados:** Es el circuito actual de vendedor viajero, s^t
- **Acción:** La selección de un par de arcos no adyacentes, e_j^t y e_k^t
- **Transición:** El cambio de estado según la elección y según la regla del Simulated Annealing, lo cual genera las probabilidades de transición (se acepta la nueva solución con cierta probabilidad)
- **Recompensa:** Es el cambio de valor entre un estado y el otro: $r_t = f(s^{t+1}) - f(s^t)$
- Con estos elementos es posible construir un sistema de RL que “aprende” a hacer mejor el SA.
- Algunos artículos recientes reportan buenos resultados en esto.

Enseñándole a una IA a “optimizar”

- Los ejemplos discutidos son casos en que técnicas de ML se usan para mejorar el rendimiento de algoritmos conocidos.
- Pero, supongamos que, aún con las mejoras, estos algoritmos tardan en llegar a buenas soluciones, para efectos del uso práctico.
- Podríamos pensar en algo muy diferente:
- Nuestro algoritmo exacto de optimización es un “experto” que puede calcular soluciones óptimas (o, suficientemente buenas).
- Tenemos los parámetros, que son los datos del problema: c .
- Supongamos que “generamos” un conjunto de variantes (o escenarios) de estos parámetros: $c_i, i = 1, \dots, r$.
- Cada uno de estos “escenarios” debería ser representativo de una situación que pueda darse.
- Ahora resolvemos el modelo para cada uno y obtenemos una familia de soluciones: $x_i^*, i = 1, \dots, r$.

Enseñándole a una IA a “optimizar”

- Ahora entrenamos una red neuronal adecuada para que “aprenda” las soluciones, a partir de los datos $\{c_i, x_i^*\}_{i=1}^r$.
- Luego, el sistema entrenado es el que se implementa en el lugar real y lo usamos para la toma real de decisiones.
- la idea es que el sistema de IA “ajuste” la predicción con las observaciones reales del “experto”



(imagen de Bengio et als(2021))

Enseñándole a una IA a “optimizar”

- Este enfoque de “aprender a optimizar imitando al experto” (imitation learning) tiene una serie de desafíos:
- La representatividad de la muestra que se le entrega a la IA.
- El manejo de restricciones es complejo y no siempre se cumplen.
- Puede ser conveniente que la estructura del problema se especifique mejor a la IA, y eso es difícil.

Enseñándole a una IA a “optimizar”

- Algunos desarrollos más recientes toman el problema desde los datos:
- Consideremos nuevamente el problema

$$\begin{aligned} z^*(c) = \min \quad & f(x, c) \\ \text{s.a.} \quad & g(x, c) \leq 0 \\ & x \text{ enteros} \end{aligned}$$

donde los c son ahora parámetros inciertos. Sea $x^*(c)$ solución óptima.

- Tradicionalmente, se usaría información histórica para estimar los parámetros c y después se usa el optimizador.
- Mandi et al (2024) proponen un esquema general en el que se busca entrenar un sistema de IA para que busque una predicción adecuada de c que, simultáneamente optimice f , ya que ese es el objetivo final...
- Esto se llama “Decision Focused Learning”, en contraste con “Predict-then-optimize”.

Enseñándole a una IA a “optimizar”

- Primero, notemos que lo relevante no siempre es tener una buena predicción de los parámetros.
- Tomemos el siguiente problema de mochila:

$$\begin{aligned} \min \quad & 3x_1 + 5x_2 + x_3 \\ \text{s.a.} \quad & x_1 + x_2 + x_3 = 2 \\ & x_j \in \{0, 1\}, j = 1, 2, 3 \end{aligned}$$

- La solución óptima es $x_1 = 1, x_2 = 0, x_3 = 1$, si todo fuera determinístico.
- Supongamos que hay incertidumbre en los coeficientes de la función objetivo.
- Y hemos estimado mal el coeficiente de x_2 , y el número que tenemos es 20 y no 5. ¿Importa?
- Lo anterior sugiere que se debe considerar el aprendizaje desde los datos hasta las soluciones y no necesariamente “gastar energía” en estimar perfectamente los parámetros del problema.

Enseñándole a una IA a “optimizar”

- Lo que se busca es construir un “predictor” $m_w(c)$, donde w son los parámetros del predictor.
- El entrenamiento se hace sobre un conjunto de “features” (variables independientes observadas) y parámetros observados, $\{z_i, c_i\}_{i=1}^N$, y se minimiza una función de pérdida (error), $\mathcal{L}(x, c)$.
- Planteamos el siguiente problema de optimización, que encuentra los mejores parámetros del “predictor”:

$$\begin{aligned} \min_w \quad & \mathcal{L}(x^*(\hat{c}), c) = \frac{1}{N} \sum_{i=1}^N \|x^*(\hat{c}_i) - x^*(c_i)\|_2^2 \\ \text{s.a.} \quad & \hat{c}_i = m_w(z_i) \quad i = 1, \dots, N \\ & x^*(\hat{c}_i) = \operatorname{argmin}\{f(x, \hat{c}_i) : g(x, \hat{c}_i) \leq 0, x \text{ entero}\} \quad i = 1, \dots, N \end{aligned}$$

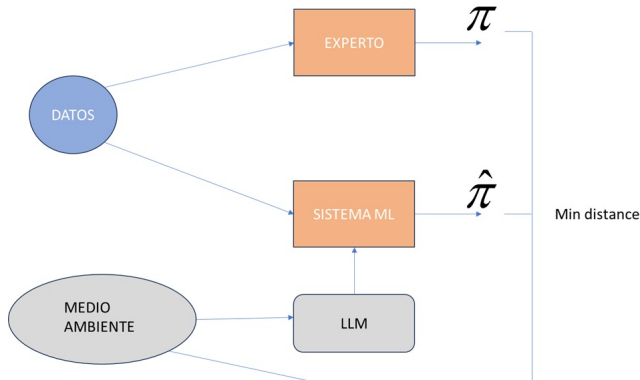
- Esto es un problema de apariencia bastante compleja...
- Cómo abordarlo eficientemente es un tema muy activo de investigación en estos últimos años.
- Notemos que, después de reemplazos, es un problema “no restringido”.
- Y podríamos usar un método basado en gradientes (o algo parecido)...

Enseñándole a una IA a “optimizar” y considerar otros elementos

- ¿Y si hay otros elementos que no se pueden representar bien en el modelo de optimización?
- Por ejemplo, supongamos estamos planificando cirugías en un hospital, para lo que hemos construido un modelo de optimización.
- Pero, al mostrar la solución a los directivos, estos dicen:
- “Se ve bien la solución pero, *a veces*, no todos los quirófanos se pueden usar por A, B, C razones”
- “Algunas personas prefieren trabajar sobretiempos *algunos* días y otros no, pero cambian de opinión frecuentemente”
- El analista querría una cuantificación precisa de “*a veces*” y “*algunos*” pero la respuesta que obtiene es “*depende*”
- Nada de esto se puede representar muy bien mediante ecuaciones, ni siquiera bajo incertidumbre.

Enseñándole a una IA a “optimizar” y considerar otros elementos

- Supongamos realizamos una encuesta, y un “grupo de foco”, al personal del hospital (el “medio ambiente” del sistema) respecto a sus preferencias y condiciones especiales respecto al problema de atender las operaciones en una semana.
- Esa encuesta entregará información adicional, pero en forma cualitativa y poco estructurada.
- Una idea: Procesar esa información con un LLM y conectarla a la IA anterior para que la combine con la solución analítica estimada.



¿Puede la IA construir (correctamente) modelos de IO?

- Modelar es una de las principales habilidades en el área de Investigación Operacional.
- El modelo es el punto de partida de todo...
- Y siempre es un desafío construir correctamente un modelo.
- En tiempos recientes se ha postulado el usar LLMs para apoyar la construcción de modelos.

Ejemplo:

- Consideremos nuevamente el problema de asignación de servidores a clientes:
- Una empresa de comunicaciones debe asignar m servidores entre n clientes.
- Hay un costo por la asignación del servidor i al cliente j : c_{ij} .
- Si el servidor i es asignado al cliente j , este obtendrá un ancho de banda igual a a_{ij} .
- cada servidor debe ser asignado a lo más a un cliente.
- Para cada cliente hay una promesa de ancho de banda mínimo igual a b_j .
- El problema es asignar todos los servidores a mínimo costo pero cumpliendo con la promesa de servicio.

Ejemplo

- Sea x_{ij} con valor 1 si se asigna el servidor i al cliente j .
- El modelo era del tipo “Asignación Generalizada”:

$$\begin{aligned} \min \quad & \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.a.} \quad & \sum_{j=1}^n x_{ij} \leq 1, \quad i = 1, \dots, m \quad (1) \\ & \sum_{i=1}^m a_{ij} x_{ij} \geq b_j, \quad j = 1, \dots, n \quad (2) \\ & x_{ij} \in \{0, 1\} \quad \forall (i, j) \end{aligned}$$

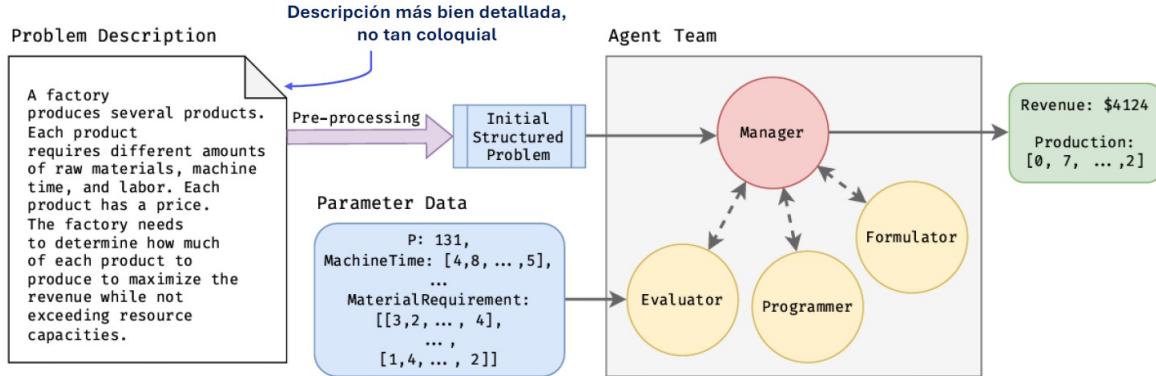
- Ahora, este mismo problema se “lo puse” a Copilot.

Ejemplo:

- Usé el siguiente “prompt” y le pedí que usara tecnología GPT5, para “analizar más”:
- Una empresa de comunicaciones debe asignar m servidores entre n clientes. Hay un costo por la asignación del servidor i al cliente j : c_{ij} . Si el servidor i es asignado al cliente j , este obtendrá un ancho de banda igual a a_{ij} . Para cada cliente hay una demanda de ancho de banda mínimo igual a b_j . Cada servidor debe asignarse a un solo cliente.
Construye un modelo de optimización que asigne servidores a clientes a mínimo costo, cumpliendo con las condiciones anteriores.
- Veamos el resultado: <https://copilot.microsoft.com/shares/wsH5vyDYRiG985yZF1bTo>

¿Pero y si la estructura es realmente compleja y la descripción no tan “formal”?

- Se han desarrollado sistemas de IA que actúan de “interface” entre el usuario y el LLM.
- Estos actúan como “traductor” de las ideas generales hacia especificaciones detalladas que, finalmente, llegan al solver.
- Por ejemplo, Teshinizi et al. (2025) han desarrollado el sistema OptiMUS.



(Figura de S. Sepúlveda, 2025, basada en Teshinizi (2025))

¿Pero y si la estructura es realmente compleja?

- El problema de la figura anterior Copilot lo puede, de hecho, abordar bien.
- Pero, el tema se pone complejo con problemas más difíciles.
- Los autores muestran resultados en los que comparan su sistema con el “caso base” enviado a GPT4.
- En el caso de problemas de estructura compleja (muchos elementos y relaciones), OptiMUS genera correctamente 67% de los problemas, mientras que ChatGPT lo hace solo con 9,5%.
- En todo caso, siempre se requiere revisión y verificación.
- Pero hay un avance y ahorro de tiempo...

Otros usos

- Hay avances, también, en otros temas igualmente considerados por nosotros como habilidades “superiores”:
- Reconocer estructuras en el problema de optimización, de modo que pueda ser abordado por una técnica como Descomposición de Benders o Relajación Lagrangiana.
- etc.

IA para reconocer estructuras

- Recordemos el modelo del problema 2 de la I2:

$$\begin{aligned} \min \quad & \sum_{t=1}^T \left\{ \sum_{j=1}^n c_{jt} x_{jt} + \sum_{j=1}^n f_{jt} y_{jt} + \sum_{i=1}^m e_{it} z_{it} + \beta_t v_t \right\} \\ \text{s.a.} \quad & \sum_{j=1}^n \alpha_{jt} x_{jt} \leq B_t, \quad t = 1, \dots, T \end{aligned} \tag{1}$$

$$x_{jt} \leq L_j y_{jt}, \quad j = 1, \dots, n; t = 1, \dots, T \tag{2}$$

$$\sum_{t=1}^T y_{jt} \leq 1, \quad j = 1, \dots, n \tag{3}$$

$$\sum_{j=1}^n x_{jt} = \sum_{i=1}^m z_{it}, \quad t = 1, \dots, T \tag{4}$$

$$\sum_{i=1}^m z_{it} \geq d_t - v_t, \quad t = 1, \dots, T \tag{5}$$

$$\sum_{t=1}^T z_{it} \leq K_i, \quad i = 1, \dots, m \tag{6}$$

$$x_{jt} \geq 0, z_{it} \geq 0, y_{jt} \in \{0, 1\}, v_t \geq 0, j = 1, \dots, n; t = 1, \dots, T; i = 1, \dots, m \tag{7}$$

IA para reconocer estructuras

- En la prueba se pedía reconocer una estructura de DW.
- Hemos destacado lo importante de reconocer estructuras.
- Ir por la estructura y/o descomposición equivocada puede ser costoso.
- Nos preguntamos si acaso una IA será capaz de reconocer estructuras adecuadamente.
- Existe investigación reciente en este tema y hay resultados, usando IA especializada.

IA para reconocer estructuras

- Le pregunté a Copilot por una estructura de Dantzig-Wolfe.
- Veamos el resultado: <https://copilot.microsoft.com/chats/mqwMnY2K8VB9X4kSd4Hc7>

¿Qué nos depara el futuro?

- Como vemos, las metodologías de ML e IA están entrando como un apoyo prometedor en la Investigación Operacional.
- El uso de ML, por ejemplo, es bastante estructurado y se puede ver muy bien cómo esas técnicas apoyan a los algoritmos, por ejemplo.
- Pero no deberíamos dejar de pensar en el potencial de la GenIA y sus sucesores...
- ¿Qué papel debe jugar en esto el analista de Investigación Operacional?
- ¿Cómo debe ser el analista de IO del futuro?